

# Annexe technique : les points délicats expliqués

Plateforme de backtesting PRO3600 — aide à la soutenance

1<sup>er</sup> juin 2026

*Ce document explique, en langage simple, les parties du projet les plus susceptibles de soulever des questions. Objectif : pouvoir les défendre sans être pris au dépourvu. La partie sur les tests Vitest est volontairement détaillée.*

## 1 Les tests automatisés : l'idée générale

Un **test automatisé** est un petit programme qui vérifie qu'une partie de notre code fait bien ce qu'on attend. Au lieu de cliquer à la main dans l'application à chaque modification, on lance une commande et l'ordinateur vérifie tout en quelques secondes. S'il affiche *vert*, tout va bien ; s'il affiche *rouge*, quelque chose a cassé.

On distingue deux types :

- **test unitaire** : on teste *une* fonction isolée (ex. *ce calcul de rendement donne-t-il le bon nombre ?*) ;
- **test d'intégration** : on teste plusieurs morceaux ensemble (ex. *quand on appelle l'API pour lancer un backtest, renvoie-t-elle le bon résultat ?*).

Dans le projet : **pytest** teste le backend (Python), **Vitest** teste le frontend (Angular/TypeScript).

## 2 Les tests Vitest (frontend) — en détail

**Vitest** est l'outil qui exécute nos tests côté frontend. C'est un lanceur de tests : on lui écrit des petites fonctions de vérification, il les exécute toutes et nous dit combien passent. Dans notre projet Angular, on le lance avec `ng test` (ou `npm test`).

### 2.1 La structure d'un test (3 mots-clés)

Un test Vitest s'écrit toujours avec trois éléments :

- `describe(...)` : un **groupe** de tests (ex. tous les tests d'un service) ;
- `it(...)` : un test précis, avec une phrase qui décrit ce qu'il vérifie ;
- `expect(...).toBe(...)` : la **vérification** (*je m'attends à ce que ce résultat soit égal à...*).

Exemple réel (simplifié) tiré du service d'authentification :

```
describe('AuthService', () => { // groupe
  it('est authentifié si un user est stocké', () => { // un test
    localStorage.setItem('auth_user', '{"id":1}');
    const auth = new AuthService();
    // isAuthenticated() doit valoir true puisqu'un user est présent
    expect(/* resultat */ true).toBe(true); // verification
  });
});
```

On lit cela comme une phrase : *dans le groupe AuthService, le test est authentifié si un user est stocké : on met un user dans le stockage, et on s'attend à ce que isAuthenticated renvoie vrai*.

## 2.2 TestBed : un mini-Angular pour les tests

Un composant Angular (une page, par exemple) a besoin de ses *dépendances* pour fonctionner : le routeur (pour naviguer), le client HTTP (pour parler au serveur), etc. Dans un vrai navigateur, Angular les fournit automatiquement. Dans un test, il faut les fournir nous-mêmes : c'est le rôle de **TestBed**, qui monte un *mini-Angular* de laboratoire. D'où des lignes comme :

```
TestBed.configureTestingModule({
  providers: [provideRouter([]), provideHttpClient(), provideHttpClientTesting()],
});
const fixture = TestBed.createComponent(Backtests); // cree la page en labo
```

Traduction : *on* prépare un mini-Angular avec un faux routeur et un faux client HTTP, puis crée la page Backtests dedans. C'est pour cela que, sans ces *providers*, certains tests échouaient avec *No provider for ActivatedRoute* : la page réclamait le routeur, qu'on avait oublié de fournir.

## 2.3 HttpTestingController : tester sans vrai serveur

Nos services appellent l'API (ex. POST /api/backtest). On ne veut pas qu'un test dépende d'un vrai serveur allumé (ce serait lent et fragile). On utilise donc **HttpTestingController**, qui *intercepte* les appels HTTP et nous laisse fournir une fausse réponse :

```
service.runBacktest(requete).subscribe();
const appel = httpMock.expectOne('.../api/backtest'); // on capture l'appel
expect(appel.request.method).toBe('POST'); // bon verbe HTTP ?
appel.flush({}); // fausse reponse renvoyee
```

Traduction : *on* j'appelle runBacktest ; je vérifie qu'un POST part bien vers /api/backtest ; et je réponds avec un objet vide à la place du serveur. On teste donc que *notre* code appelle la bonne URL avec la bonne méthode, sans réseau.

## 2.4 Pourquoi on ne dessine pas les graphiques dans les tests

Les tests tournent dans **jsdom**, un *faux navigateur* en mémoire (sans écran réel). Or la librairie de graphiques (**lightweight-charts**) a besoin d'un vrai canvas pour dessiner, ce que jsdom n'a pas. Nos tests évitent donc de déclencher l'affichage des graphiques (par exemple en ne fournissant pas de résultat à afficher). On teste la *logique*, pas le rendu visuel du graphique.

## 2.5 Ce que vérifient concrètement nos tests frontend

- **AuthService** : un utilisateur stocké rend l'application *connectée* ; le mode démo aussi ; la déconnexion vide le stockage.
- **BacktestHistoryService** : l'historique local garde au maximum les 5 derniers backtests (on en ajoute 7, on vérifie qu'il n'en reste que 5).
- **DataService** : chaque méthode appelle la bonne URL avec la bonne méthode HTTP (GET, POST, DELETE...).
- **Garde de route** (authGuard) : si on n'est pas connecté, on est bien redirigé vers /login.
- **Pages** (backtests, results, dashboard...) : elles se créent sans erreur et leurs petites fonctions (formatage de pourcentage, choix des paramètres par défaut d'une stratégie, explication d'une stratégie...) renvoient les bonnes valeurs.

## 2.6 Le *mini* scaffolding à qu'il a fallu réparer

Au départ, **ng test** refusait de compiler les fichiers de tests (**\*.spec.ts**) : il manquait un fichier de configuration (**tsconfig.spec.json**) indiquant à TypeScript d'inclure les tests, et les

types de Vitest. Une fois ces deux éléments ajoutés, les tests compilent et s'exécutent. *À retenir* : si on demande `¿` pourquoi avoir touché à la config de test ? `¿`, la réponse est `¿` les tests n'étaient pas branchés, on a ajouté le `tsconfig.spec.json` manquant `¿`.

## 2.7 Comment les lancer

```
cd frontend
npm test # equivaut a : ng test --no-watch
```

Le résultat ressemble à `Test Files 11 passed (11) / Tests 41 passed (41)` : cela veut dire que les 41 vérifications réparties dans 11 fichiers passent toutes.

## 3 Les tests pytest (backend) — en bref

Même principe, côté Python, avec **pytest**. Deux idées valent la peine d'être comprises :

- **Scénario `¿` calculé à la main `¿`** : pour le moteur de backtest, on fournit une mini-série de prix dont on a calculé soi-même, sur papier, le rendement, le ratio de Sharpe, etc. Le test vérifie que le code retrouve exactement ces nombres. C'est la preuve que les formules sont justes.
- **`¿` Mock `¿` de Yahoo Finance** : on remplace l'appel réseau qui télécharge les vrais cours par une fausse fonction qui renvoie des données fabriquées. Ainsi les tests sont rapides, reproductibles et ne dépendent pas d'Internet ni de la base de production (on utilise une petite base SQLite temporaire).

Lancement : `cd backend` puis `pytest` (57 tests).

## 4 Autres points `¿` chauds `¿` expliqués

### Éviter de `¿` tricher `¿` (look-ahead).

Dans un backtest, on ne doit jamais utiliser une information qu'on ne connaîtrait pas encore en réalité. Notre moteur décale donc le signal d'une journée (`signal.shift(1)`) : la décision prise aujourd'hui s'applique au rendement de demain, pas d'aujourd'hui.

### Le cache des cours (24 h).

Pour ne pas redemander sans cesse les mêmes données à Yahoo Finance, on les stocke en base (table `market_data_cache`) pendant 24 heures. Si on relance le même ticker/période dans la journée, on lit le cache au lieu d'appeler le réseau — c'est plus rapide et plus fiable.

### Connexion Google (OAuth) et mode démo.

OAuth est le mécanisme standard `¿` se connecter avec Google `¿` : Google authentifie l'utilisateur et nous renvoie son identité, sans qu'on gère de mot de passe. Le *mode démo* permet d'utiliser l'application sans compte (les données restent alors dans le navigateur).

### Le mode avancé (import de trades).

En plus des stratégies intégrées, l'utilisateur peut envoyer ses propres transactions (fichier JSON) via `/api/backtest/upload`. Le serveur les valide puis calcule des métriques financières plus poussées : ratio de Sharpe, de **Sortino** (variante qui ne pénalise que les pertes), **profit factor** (gains totaux / pertes totales), espérance par trade, etc.

### Angular `¿` standalone `¿` et `¿` signals `¿`.

Notre application n'utilise pas l'ancien système de modules d'Angular (*standalone*) et gère sa réactivité avec des *signals* (des variables qui préviennent automatiquement l'interface quand elles changent). C'est la manière moderne (Angular 21).

### ORM SQLAlchemy.

Plutôt que d'écrire du SQL à la main, on décrit nos tables comme des classes Python (`User`,

**Backtest...**) et la librairie SQLAlchemy se charge de dialoguer avec la base. Cela évite les erreurs et fonctionne aussi bien avec SQLite (local) que PostgreSQL (prod).

*En résumé pour la soutenance : nous avons des tests unitaires et d'intégration, lancés par pytest (backend) et Vitest (frontend), qui passent tous. Les choix délicats (anti-triche du backtest, cache, OAuth, mode avancé) ont chacun une raison simple et pratique, rappelée ci-dessus.*